Sistemas Industriales Inteligentes

Estructura, objetivos y proyecto del curso

Antonio Miguel López Rodríguez
Máster en Ingeniería de Automatización e Informática Industrial
Curso 2026–2027

Estructura del curso

Aprenderemos a **comprender, construir y evaluar sistemas que apoyan decisiones industriales**.

Ocho semanas de trabajo progresivo

Cada semana incorpora una técnica, ejemplos y práctica con datos.

El proyecto común reúne lo aprendido; comparamos alternativas y sus límites.

Un periodo para el proyecto final

Después habrá aproximadamente un mes de trabajo autónomo.

Aplicarás las técnicas y justificarás tus decisiones en un problema concreto.

La primera semana nivela herramientas y lectura de datos. Las siguientes amplían el trabajo con modelos, agentes y optimización.

Recorrido por semanas

Semana	Qué aprenderemos
1 · Fundamentos	Preparar herramientas, entender los datos y construir una tabla horaria.
2 · Regresión	Estimar una variable continua y evaluar con orden temporal.
3 · Clasificación	Detectar problemas de calidad y valorar errores de decisión.
4 · No supervisado	Explorar estructura y monitorizar el proceso.
5 · Redes y secuencias	Comparar modelos que utilizan la evolución temporal.
6 · Despliegue	Ofrecer el modelo como servicio, vigilarlo y explicar sus resultados.
7 · Agentes	Combinar reglas, modelos y herramientas para recomendar.
8 · Optimización	Buscar ajustes y seleccionar variables bajo restricciones.

RITMO SEMANAL DESDE S2

Cómo trabajaremos



ENTREGA: CUADERNO EJECUTADO + INFORME BREVE → DEFENSA ORAL

S1 es nivelación sin defensa. La primera defensa formal es la tarea de S2, al abrir S3.

EVALUACIÓN

Cómo se califica

10% | **Trabajo continuo**

Tareas ancla semanales evaluadas mediante defensa oral individual, no mediante corrección automática de la entrega.

40% | **Examen**

Orientado a diagnosticar, interpretar y criticar resultados, no a resolver problemas.

50% | **Proyecto final**

Aplicar las técnicas a un problema, comparar alternativas y justificar las decisiones. Trabajo individual o en grupo.

El proyecto final

Vía A · Proyecto ancla

Continuar el proyecto común y desarrollar una de sus líneas.

La elección de la línea se prepara en las semanas 7–8.

Vía B · Proyecto propio

Aplicar las técnicas a un problema viable de tu interés.

Propuesta de una página y aprobación hacia las semanas 3–4.

Trabajo individual o en grupo. Tras las ocho semanas habrá aproximadamente un mes de trabajo autónomo.

Qué entendemos por IA industrial

Sistemas que utilizan **datos y conocimiento** para interpretar situaciones y apoyar decisiones en un entorno industrial.

Observar e interpretar

Percibir el entorno, reconocer estados, diagnosticar problemas y anticipar lo que puede ocurrir.

Ejemplos: inspección de piezas, detección de fallos y predicción de calidad.

Planificar y decidir

Comparar alternativas, organizar operaciones y proponer acciones sujetas a restricciones.

Ejemplos: programar producción, elegir una ruta o recomendar ajustes.

La solución debe encajar en un proceso físico, con objetivos operativos y personas que utilizan sus resultados.

Más allá del aprendizaje automático

El aprendizaje con datos es una de las herramientas que puede utilizar un sistema inteligente.

Enfoque	Qué aporta
Reglas y conocimiento	Expresar criterios de expertos y razonar con condiciones explícitas.
Búsqueda y optimización	Comparar alternativas y buscar soluciones que respeten restricciones.
Aprendizaje automático	Aprender relaciones a partir de ejemplos para estimar, clasificar o detectar patrones.
Agentes	Combinar conocimiento, modelos y herramientas para resolver una tarea.

Estas herramientas pueden combinarse con modelos físicos, métodos estadísticos y sistemas de automatización ya existentes.

Dónde se aplica la IA industrial

Ámbito	Ejemplo de problema
Calidad	Inspeccionar productos o estimar su composición.
Mantenimiento	Detectar indicios de fallo y priorizar intervenciones.
Energía	Prever demanda y proponer una operación más eficiente.
Producción	Planificar tareas y asignar recursos.
Logística	Organizar rutas y movimientos de materiales.
Robótica	Interpretar el entorno y planificar movimientos.

La técnica se elige por el problema, los datos, las restricciones y el uso previsto del resultado.

Por qué usaremos un mismo problema

Trabajaremos con datos reales de una **planta de flotación de mineral de hierro**.

Un caso para conectar técnicas

Permite preparar datos, estimar calidad, detectar anomalías y estudiar la evolución temporal.

Después incorporamos despliegue, recomendaciones y búsqueda de ajustes.

Una comparación que se acumula

Reutilizamos el trabajo anterior y comprobamos qué aporta cada técnica nueva.

Así construimos un sistema progresivamente y aprendemos también de los resultados modestos.

Los ejemplos satélite usan otros datos y problemas para ampliar la perspectiva y entender cada técnica.

El problema industrial: separar hierro y sílice

El mineral que llega a la planta contiene **minerales de hierro mezclados con sílice** (cuarzo, SiO_2).

La alimentación

La proporción de hierro y de sílice puede cambiar con el mineral que llega de la mina.

La planta debe separar esa mezcla mediante un proceso continuo.

El producto: concentrado

Queremos un producto enriquecido en hierro y con poca sílice residual.

El porcentaje de sílice del concentrado permite valorar la calidad de la separación.

El problema que guía el curso es conocer y vigilar cuánta sílice permanece en el producto final.

La flotación reversa del mineral de hierro

El mineral molido se mezcla con agua para formar una **pulpa**. Se añaden reactivos y se inyecta aire: las burbujas ayudan a separar las partículas.

La sílice sube con la espuma

Los reactivos favorecen que sus partículas se adhieran a las burbujas.

La espuma se retira por la parte superior, arrastrando sílice.

El hierro queda en el concentrado

Se busca que las partículas de hierro permanezcan en la pulpa.

La corriente de concentrado recoge el producto que interesa conservar.

Se llama reversa porque se hace flotar la impureza que queremos retirar. La separación puede dejar sílice en el concentrado.

Reactivos y condiciones de separación

La separación depende de cómo interactúan el mineral, el agua, los reactivos y las burbujas.

Variable	Papel en el proceso
Amina	Favorece que la sílice se adhiera a las burbujas y salga con la espuma.
Almidón	Ayuda a mantener las partículas de hierro mojables, para que permanezcan en el concentrado.
pH de la pulpa	Mantiene el medio alcalino en el que actúan los reactivos.
Aire y nivel	Describen las condiciones de funcionamiento de las columnas de flotación.

Los caudales de reactivos son ajustes de operación. Sus asociaciones con la calidad deben interpretarse junto con el estado del proceso.

Qué sabemos de la planta y de la calidad

El proceso se observa cada 20 s

Se registran caudales de reactivos y pulpa, pH, densidad, aire y nivel de siete columnas.

Estas señales permiten seguir cambios de operación dentro de una hora.

La calidad viene del laboratorio

El hierro y la sílice del concentrado tienen resolución horaria en los datos.

El operador dispone de menos información sobre cómo evoluciona la calidad entre análisis.

Queremos estimar la sílice a partir de las señales de proceso, contrastar esa estimación con el laboratorio y apoyar las decisiones del operador.

Nuestro proyecto: un sensor virtual

Lo que podemos medir

Caudales, reactivos, pH, densidad, aire y nivel de las columnas.

Son señales del funcionamiento de la planta.

Lo que queremos estimar

El porcentaje de sílice en el concentrado, que se mide en laboratorio.

Un *soft sensor* aprende esa relación a partir de los datos.

El curso ampliará esta estimación con monitorización, recomendaciones a un operador humano y optimización.

01

BLOQUE DE CONSULTA

Recursos y detalle por semanas

Recursos de trabajo, mapa técnico y fichas de cada semana: formulación, algoritmos, extractos reales y resultados para consultar durante el curso.

MATERIALES

Entorno técnico y formato del material

Stack

Python con numpy, pandas, matplotlib, scipy, scikit-learn, TensorFlow/Keras, SHAP, FastAPI, el SDK `anthropic` y MCP. Versiones fijadas en `requirements.txt`.

Cuadernos

`Python_minimo`: capa nueva de librerías, de consulta. `ALUMNO`: con TODO. `SOLUCION`: ejecutados de punta a punta con salidas reales.

Enunciados

S2–S8: contexto, reglas, *floor/ceiling*, defensa oral y criterios. S1: práctica guiada, sin enunciado formal ni defensa.

Entorno

Local con entorno virtual (con guía) o Google Colab (guía para la semana 1). Datasets pequeños en el repositorio; los grandes de la S5, aparte.

VISIÓN DE CONJUNTO

Mapa del curso

Sem.	Técnicas	Satélites (datos)	Tarea ancla
1	pandas, eje temporal, muestreo mixto, agregación horaria	Subconjunto real de 112 h	Tabla horaria curada
2	Regresión lineal, ecuación normal, descenso de gradiente, ridge/lasso, CV temporal	Datos sintéticos	*Soft sensor* de % sílice
3	Logística, árboles, ensembles, ROC/AUC, umbral, multiclase	Juguete 2D, vinos UCI	Control de calidad (fuera de spec)
4	k-means, GMM, DBSCAN, PCA, MSPC (T^2 y Q)	2D de 4 clases, proceso sintético, Iris	Regímenes + monitorización
5	MLP, backprop, Adam, CNN, transfer learning, LSTM/GRU, 1D-CNN	Serie sintética, dígitos, flores	*Soft sensor* secuencial a 20 s
6	FastAPI, MLOps, drift, SHAP, AI Act, IA generativa	— (directo sobre el ancla)	Empaquetar, vigilar, explicar
7	Agentes de reglas, *tool-use*, patrón ReAct	Percentiles del proceso, *soft sensor*	Agente supervisor que recomienda
8	GA, Nelder-Mead, evolución diferencial, selección de variables	Rosenbrock, Schaffer F6, viajante	Consignas + subconjunto de variables

SEMANA 1 · FUNDAMENTOS Y DATOS DEL PROYECTO

Del dato crudo a la tabla horaria

QUÉ SE TRABAJA

- Python y pandas mínimos: selección de ejemplos en la primera sesión y resto como consulta. Incluye funciones propias (`def` , `return`), utilizadas desde S2.
- **Eje temporal**: fechas repetidas ~180 veces por hora; horas completas e incompletas.
- **Muestreo mixto**: variables rápidas a 20 s frente a laboratorio horario; qué agregar y cómo.
- **Enventanado** y agregación horaria; variables derivadas (aire y nivel medios de las 7 columnas).
- Opción de hacerlo en Google Colab con el CSV pequeño.

PROYECTO ANCLA

- Construir la tabla horaria curada a partir de un subconjunto real de 112 h (10–14 de marzo de 2017).
- La misma mecánica sobre el crudo completo (184 MB) produce las 4.097 h que usan las semanas 2–8.
- Sin enunciado formal ni defensa: nivelación con propósito. El ALUMNO hace de tarea: 9 bloques TODO cortos, uno con seis asignaciones, y ampliación opcional (estadística, gráficas, comprobaciones).

Sesiones: herramientas (2 h), proyecto/datos (1 h), ALUMNO (2 h). Plan de trabajo integrado en la presentación específica de S1.

SEMANA 1 · DETALLE TÉCNICO

Agregación horaria: un estadístico distinto para cada variable

DECISIONES DE AGREGACIÓN

- Clave de agrupación: `date`, la marca horaria que se repite ~180 veces por hora.
- **Variables rápidas** (reactivos, caudal, pH, densidad, aire, nivel): media horaria. Aire y nivel se promedian antes sobre las 7 columnas de flotación.
- **% Fe y % SiO_2 de alimentación**: se verifica que son constantes dentro de la hora y se toma `first`.
- **Sílice (objetivo)**: `mean`, no `first`, porque en algunas horas es una rampa interpolada y `first` se quedaría con un extremo arbitrario.
- `n_muestras` = tamaño del grupo: marca las horas incompletas.
- La ampliación comprueba por hora cuántos valores distintos tiene cada variable (`nunique`, mín., máx.).
- El alumno construye la tabla con `groupby().mean()` columna a columna; la forma `.agg()` y las comprobaciones son ampliación.

SEMANA1_SOLUCION.IPYNB · EXTRACTO

```
datos['aire_medio'] = datos[cols_aire].mean(axis=1)
datos['nivel_medio'] = datos[cols_nivel].mean(axis=1)
medias = datos.groupby('date', as_index=False).mean()
primeros = datos.groupby('date', as_index=False).first()
filas = datos.groupby('date', as_index=False).size()
tabla = pd.DataFrame()
tabla['date'] = medias['date']
tabla['n_muestras'] = filas['size']
tabla['silice'] = medias['% Silica Concentrate']
tabla['iron_feed'] = primeros['% Iron Feed']
tabla['silica_feed'] = primeros['% Silica Feed']
tabla['amina_media'] = medias['Amina Flow']
tabla['aire_medio'] = medias['aire_medio']
tabla['nivel_medio'] = medias['nivel_medio']
```

SEMANA 2 · SUPERVISADO I: REGRESIÓN

El *soft sensor* y la validación honesta

QUÉ SE TRABAJA

- Capa nueva: numpy, matplotlib y el patrón `fit` / `predict` de scikit-learn.
- Satélite sobre datos sintéticos: regresión lineal y **ecuación normal**; **descenso de gradiente programado desde cero**; ingeniería de características; selección de modelo y validación cruzada; **ridge y lasso**.
- Anexo de "higiene industrial": fuga de datos, partición temporal, baseline y persistencia.
- Teoría: regresión lineal (46 diapositivas) y derivada numérica.

PROYECTO ANCLA: *SOFT SENSOR* DE % SÍLICE

- Entradas: solo las 9 variables de proceso. Prohibidas `silice`, `fuera_spec`, `date` y `n_muestras`.
- **Partición temporal 80/20**; estandarización ajustada solo con train.
- *Floor*: baseline (media) y un modelo lineal con MAE, RMSE y R^2.
- *Ceiling*: CV temporal frente a aleatoria; regularización con variables expandidas; desfases, medias móviles, última sílice de laboratorio y tabla horaria enriquecida; comparación con la persistencia.

Entrega: cuaderno ejecutado + informe de media página + defensa oral. Un R^2 bajo no penaliza si el diagnóstico es honesto.

SEMANA 2 · DETALLE TÉCNICO (1/2)

Mínimos cuadrados: forma cerrada y descenso de gradiente

FORMULACIÓN

$$J(\beta) = (1/n)\,\|y - X\beta\|^2$$

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad \text{ecuación normal (pseudoinversa)}$$

$$\nabla J = (1/n)\,X^T(X\beta - y) \qquad \beta \leftarrow \beta - \alpha\,\nabla J$$

- El satélite usa datos sintéticos con coeficientes conocidos: se comprueba que ambos métodos recuperan la "verdad oculta".
- Las dos entradas tienen escalas muy distintas a propósito: con el mismo α = 0,3, el descenso converge con las variables estandarizadas y se dispara sin estandarizar.
- Después, el mismo problema con `LinearRegression` de scikit-learn: el patrón `fit` / `predict`.

SEMANA2_SOLUCION.IPYNB · EXTRACTO

```
from numpy.linalg import pinv
# forma cerrada
beta_fn = pinv(X.T @ X) @ X.T @ y
# descenso de gradiente vectorizado (MSE)
def descenso_gradiente(X, y, alfa, n_iter):
    n, p = X.shape
    beta = np.zeros(p)
    historial = np.zeros(n_iter)
    for k in range(n_iter):
        y_est = X @ beta
        historial[k] = np.mean((y - y_est)**2)
        grad = (X.T @ (y_est - y)) / n
        beta = beta - alfa * grad
    return beta, historial
```

SEMANA 2 · DETALLE TÉCNICO (2/2)

Regularización y validación que respeta el tiempo

REGULARIZACIÓN

$$\text{Ridge: } J(\beta) + \lambda \|\beta\|_2^2 \quad \text{Lasso: } J(\beta) + \lambda \|\beta\|_1$$

- Ridge encoge los coeficientes; lasso además anula algunos, así que selecciona variables.
- Se ajustan sobre variables estandarizadas, con el escalador ajustado **solo** con train.

VALIDACIÓN

- `KFold(shuffle=True)` mezcla horas contiguas, casi idénticas, entre train y validación.
- `TimeSeriesSplit` valida siempre sobre horas posteriores a las de entrenamiento.
- Dos referencias obligatorias: el baseline ŷ = media de train y la persistencia ŷ(t) = y(t−1).

SEMANA2_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
from sklearn.model_selection import KFold, TimeSeriesSplit
def r2_fold(ti, vi):
    sc = StandardScaler().fit(X_all[ti])   # solo train
    modelo = Ridge(10).fit(sc.transform(X_all[ti]), y_all[ti])
    return r2_score(y_all[vi],
                    modelo.predict(sc.transform(X_all[vi])))
r_rand = [r2_fold(ti, vi) for ti, vi in
          KFold(5, shuffle=True, random_state=0).split(X_all)]
r_temp = [r2_fold(ti, vi) for ti, vi in
          TimeSeriesSplit(5).split(X_all)]
# misma función, mismo modelo: solo cambia el esquema
# de partición, y con él la conclusión
```

SEMANA 3 · SUPERVISADO II: CLASIFICACIÓN

Control de calidad y un marco de evaluación completo

QUÉ SE TRABAJA

- **Regresión logística programada desde cero**; cuaderno opcional que deriva el gradiente con la regla de la cadena y lo comprueba numéricamente.
- Clasificadores lineales, **árboles de decisión**, Random Forest y Gradient Boosting.
- Evaluación: matriz de confusión, precisión, recall, F1, **ROC/AUC** y umbral según el coste de cada error.
- Satélite **multiclase** (vinos UCI, 7 clases): matriz K×K, F1 macro/weighted/micro, recall por clase, métricas ordinales y cambio de dominio blancos → tintos.

PROYECTO ANCLA: ¿HORA FUERA DE ESPECIFICACIÓN?

- Etiqueta `fuera_spec` = sílice > 3,0 %. Partición temporal 80/20: 3.277 h de train y 820 de test, con un 23,0 % de positivos en test.
- Clases desbalanceadas: pesos de clase balanceados y umbral razonado.
- Comparar baseline mayoritario, logística y ensembles por F1 y recall de la clase minoritaria y por AUC.
- Punto de bifurcación: Vía A (ancla guiada) o Vía B (proyecto propio, con propuesta de una página).

Materiales: satélite de mecánica · satélite multiclase · deck de evaluación · decks de discriminantes, árboles y métodos agregados

SEMANA 3 · DETALLE TÉCNICO (1/2)

Regresión logística programada desde cero

FORMULACIÓN

$$h = \sigma(Xa), \quad \sigma(z) = 1 / (1 + e^{-z})$$

$$J(a) = -(1/n) [y^T \log h + (1 - y)^T \log(1 - h)]$$

$$\nabla J = (1/n) X^T(h - y)$$

- El gradiente tiene la misma forma que en regresión lineal, cambiando Xβ por σ(Xa); el cuaderno opcional lo deriva con la regla de la cadena y lo comprueba numéricamente.
- Se normalizan las variables para que el descenso converja y se deshace la normalización para interpretar los coeficientes.
- Con umbral 0,5 la frontera de decisión es lineal en las variables; los árboles y ensembles se introducen como alternativa no lineal.

SEMANA3_CLASIFICACION_SOLUCION.IPYNB · EXTRACTO

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
def coste_logistico(a, X, y):
    nf = X.shape[0]
    h = sigmoid(X @ a)
    eps = np.finfo(float).eps        # evita log(0)
    j = -(1 / nf) * (y.T @ np.log(h + eps)
                     + (1 - y).T @ np.log(1 - h + eps))
    grad = X.T @ (h - y) / nf
    return float(np.squeeze(j)), grad
def normaliza(X):
    m, s = X.mean(0), X.std(0)
    Xn = X.copy()
    for k in range(X.shape[1]):
        if s[k] != 0:
            Xn[:, k] = (X[:, k] - m[k]) / s[k]
    return Xn, m.reshape(-1, 1), s.reshape(-1, 1)
```

SEMANA 3 · DETALLE TÉCNICO (2/2)

Evaluar un clasificador: métricas, umbral y multiclase

BINARIO

$$P = TP / (TP + FP) \quad R = TP / (TP + FN)$$

$$F1 = 2PR / (P + R)$$

- **ROC**: tasa de verdaderos frente a falsos positivos al barrer el umbral. El **AUC** no depende del umbral; precisión, recall y F1 sí.
- **Umbral**: se elige por el coste relativo de dejar escapar una hora mala frente a dar una falsa alarma, no por defecto en 0,5.
- **Desbalanceo**: `class_weight='balanced'` pondera cada clase de forma inversamente proporcional a su frecuencia.

MULTICLASE Y ORDINAL (VINOS, K = 7)

- Matriz de confusión K×K y recall por clase, para ver las clases que el modelo nunca predice.
- **F1 macro**: media simple de las K clases. **Weighted**: ponderada por soporte. **Micro**: global, coincide con la accuracy.
- **Ordinal**: error absoluto medio en número de clases y acierto con tolerancia ±1, porque confundir un 5 con un 6 no es lo mismo que con un 9.
- **Cambio de dominio**: el modelo entrenado con vinos blancos se aplica sin cambios a tintos.

SEMANA 4 · NO SUPERVISADO

Regímenes de operación y monitorización multivariante

QUÉ SE TRABAJA

- Lab A, clustering sobre un 2D de 4 clases: **k-means con silhouette**, **GMM con BIC/AIC** y **DBSCAN**.
- Lab B, PCA y MSPC: proceso sintético con factores latentes y fallos inyectados, más Iris. **T^2 de Hotelling frente a SPE/Q** y *contribution plot*.
- Anexo que explica cómo se generan los datos sintéticos y por qué T^2 y Q detectan fallos distintos.
- Teoría: clustering y PCA como proyección ortogonal de máxima varianza.

PROYECTO ANCLA

- 9 variables estandarizadas. k-means con k = 2…8, elegido por silhouette (no por el codo). PCA y *scree*.
- **MSPC** ajustado sobre las horas en especificación: A = 5 componentes; límites en el percentil 99 empírico de T^2 y de Q.
- *Ceiling*: GMM + BIC, barrido de eps en DBSCAN, *contribution plot* de una alarma y validación temporal del MSPC.
- "Gloria", sin solución de referencia: ¿el límite es el MSPC o el problema? Probar otro detector de anomalías.

Pregunta que guía la semana: ¿la estructura no supervisada del proceso se relaciona con la calidad del producto?

SEMANA 4 · DETALLE TÉCNICO (1/2)

MSPC: PCA, T² de Hotelling y Q (SPE)

FORMULACIÓN (A = 5 COMPONENTES)

$$t = P^T x \quad T^2 = \Sigma_a \, t_a^2 / \lambda_a$$

$$Q = \|x - P \, P^T \, x\|^2$$

- La PCA se ajusta **solo con las horas en especificación** (operación normal), con las 9 variables estandarizadas con esa misma referencia.
- **T²**: desplazamiento grande dentro del plano del modelo. **Q**: rotura de la estructura de correlación, lo que el modelo no sabe reconstruir.
- Límites de control: percentil 99 empírico de T² y de Q en operación normal.
- *Contribution plot*: el residuo al cuadrado de cada variable señala cuál dispara Q.

TAREA_SEMANA4_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
normal = oos == 0                       # horas en spec
sc_n = StandardScaler().fit(X[normal])
Xn, Xa = sc_n.transform(X[normal]), sc_n.transform(X)
A = 5
pm = PCA(A).fit(Xn); lam = pm.explained_variance_
def t2_spe(Z):
    T = pm.transform(Z)
    t2 = np.sum(T**2 / lam, axis=1)
    q = np.sum((Z - pm.inverse_transform(T))**2, axis=1)
    return t2, q
T2, Q = t2_spe(Xa); T2n, Qn = t2_spe(Xn)
UCL_T2 = np.percentile(T2n, 99)
UCL_Q  = np.percentile(Qn, 99)
# AUC de T² y Q frente a fuera_spec:
# ¿anomalía de proceso = mala calidad?
```

SEMANA 4 · DETALLE TÉCNICO (2/2)

Clustering: elegir el número de grupos sin hacer trampas

CRITERIOS

$$s(i) = (b - a) / \max(a, b)$$

a: distancia media a su propio cluster; b: al cluster vecino más cercano. Por debajo de ~0,25 no hay estructura clara.

$$BIC = -2 \log L + p \log n$$

Para GMM: penaliza el número de parámetros p. Se busca un mínimo claro al aumentar el número de componentes.

PROCEDIMIENTO EN EL ANCLA

- k-means con k = 2…8 sobre las 9 variables estandarizadas; k por silhouette, no por el codo.
- Perfil de cada cluster: sílice media y % de horas fuera de spec, que se miran **a posteriori** (nunca son entrada).
- GMM con covarianza completa, de 1 a 8 componentes, y selección por BIC.
- DBSCAN: barrido de eps; número de clusters y % de puntos marcados como ruido.
- Lab A, sobre 2D de 4 clases, para ver las tres técnicas donde la estructura sí existe.

SEMANA 5 · DEEP LEARNING

Soft sensor secuencial sobre la señal a 20 s

QUÉ SE TRABAJA

- Capa de Keras (cuaderno de consulta).
- Lab A: **MLP desde cero**; el fallo de MSE con inicialización grande frente a entropía cruzada con inicialización pequeña. **LSTM frente a GRU** en una serie sintética (MAE, R^2, nº de parámetros).
- Lab B: **CNN 2D** frente a MLP plano en dígitos manuscritos, más regresión del ángulo de rotación.
- Lab C (opcional): VGG16 sin reentrenar y **transfer learning** en flores.
- Teoría: backprop, SGD con momento, RMSProp, Adam, CNN, RNN y atención o transformers (conceptual).

PROYECTO ANCLA, EN TRES NIVELES

- Dato crudo completo: 180 pasos de 20 s por hora.
- *Floor* (estimación de la hora actual): OLS de S2 como control; "puente" con 45 estadísticos intra-hora (media, desviación, mínimo, máximo y pendiente de 9 variables) + Ridge; red pequeña (1D-CNN o LSTM/GRU) sobre los 180 pasos, con *early stopping* en validación temporal.
- *Ceiling* 1 (predicción de la hora siguiente): ventanas de $k \in \{1, 3, 6, 12, 24, 48\}$ h y baseline AR(k) obligatorio. Se mide la mejora (delta) al añadir el proceso.
- *Ceiling* 2 ("gloria"): lo mismo con el proceso denso a 20 s.

Es la única semana que necesita ficheros grandes fuera del repositorio: crudo completo (184 MB), dígitos y flores.

SEMANA 5 · DETALLE TÉCNICO (1/2)

Redes sobre la secuencia cruda: 180 pasos × 9 variables

MONTAJE DEL EXPERIMENTO

- Entrada: tensor (horas, 180, 9), normalizado por variable con medias y desviaciones de train.
- Validación: el último 15 % de train, en orden temporal; test: el 20 % final.
- Entrenamiento: Adam (lr = $5 \cdot 10^{-4}$), pérdida MSE, lotes de 64, *early stopping* con paciencia 15 y recuperación de los mejores pesos.
- Redes deliberadamente pequeñas, con dropout y L2: hay ~3.300 horas de entrenamiento.
- Control obligatorio: el OLS horario de la S2 sobre la misma partición.

SEMANA5_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
cnn = keras.Sequential([
    keras.Input(shape=(180, 9)),
    layers.Conv1D(16, 7, padding="same", activation="relu"),
    layers.Dropout(0.3), layers.MaxPooling1D(2),
    layers.Conv1D(32, 5, padding="same", activation="relu"),
    layers.Dropout(0.3), layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation="relu",
                 kernel_regularizer=keras.regularizers.l2(1e-3)),
    layers.Dense(1),
])
gru = keras.Sequential([
    keras.Input(shape=(180, 9)),
    layers.GRU(16, dropout=0.2),
    layers.Dense(8, activation="relu",
                 kernel_regularizer=keras.regularizers.l2(1e-3)),
    layers.Dense(1),
])
modelo.compile(optimizer=keras.optimizers.Adam(5e-4), loss="mse")
parada = keras.callbacks.EarlyStopping(monitor="val_loss",
            patience=15, restore_best_weights=True)
```

SEMANA 5 · DETALLE TÉCNICO (2/2)

El "puente" y la pregunta del *ceiling*: ¿aporta algo el proceso?

PUENTE (*FLOOR*)

Cada hora se resume con 5 estadísticos de cada una de las 9 variables (45 columnas) y se ajusta un Ridge. La pendiente es la de mínimos cuadrados frente al tiempo centrado:

$$m = \Sigma\,(x_t - \bar{x})(t - \bar{t}) \,/\, \Sigma\,(t - \bar{t})^2$$

VENTANAS DE K HORAS (*CEILING* 1)

$$\Delta(k) = R^2_{\text{proceso + AR(k)}} - R^2_{\text{AR(k)}}$$

AR(k): sílice(t+1) a partir de las k sílices anteriores (Ridge). Proceso + AR: lo mismo más el resumen de proceso de esas k horas, aplanado. Barrido k ∈ {1, 3, 6, 12, 24, 48}.

SEMANA5_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
def resumen_intra_hora(X):          # X: (horas, 180, 9)
    media, desv = X.mean(axis=1), X.std(axis=1)
    minimo, maximo = X.min(axis=1), X.max(axis=1)
    L = X.shape[1]
    t = np.arange(L, dtype=np.float32); t = t - t.mean()
    Xc = X - X.mean(axis=1, keepdims=True)
    pendiente = np.tensordot(Xc, t, axes=([1], [0])) \
                / (t ** 2).sum()
    return np.concatenate([media, desv, minimo, maximo,
                           pendiente], axis=1)   # 45 columnas
# ceiling 1, para cada k
ar = Ridge(1.0).fit(sc_ar.transform(Xs_tr), y_tr)
r2_ar = r2_score(y_te, ar.predict(sc_ar.transform(Xs_te)))
Xflat_tr = np.concatenate([Xp_tr.reshape(k80w, -1), Xs_tr],
                          axis=1)
ridge_pf = Ridge(10.0).fit(sc_pf.transform(Xflat_tr), y_tr)
```

SEMANA 6 · IA INDUSTRIAL EN PRODUCCIÓN

Del cuaderno al sistema

QUÉ SE TRABAJA

- No se entrena ningún modelo nuevo: el *soft sensor* de la S2 pasa de cuaderno a sistema.
- **Despliegue y MLOps**: de DevOps a MLOps, ciclo de vida, entrenamiento continuo, registro de modelos y contenedores.
- **Drift** y degradación silenciosa; explicabilidad con **SHAP**; **AI Act** (pirámide de riesgo); IA generativa.
- Cuaderno ALUMNO con el código ya resuelto y 4 tareas ligeras ("Tu turno" / "Interpreta").
- Demo opcional: la misma función expuesta como servidor **MCP** para clientes como Claude Desktop.

PROYECTO ANCLA

- **F1 Empaquetar**: función servible `predict_silice` y API con FastAPI sobre el modelo serializado (joblib).
- **F2 Vigilar**: el MSPC de la S4, ajustado solo con train, como monitor de drift sobre test.
- **F3 Explicar**: SHAP de una predicción concreta y ranking global de importancia.
- **F4 Situar**: clasificar el sistema según el AI Act.
- *Ceiling*: informe de planta generado por un LLM y revisado críticamente, política de reentrenamiento y arquitectura edge / planta / nube.

Semana deliberadamente más ligera que la anterior.

SEMANA 6 · DETALLE TÉCNICO (1/2)

Empaquetar: el modelo como servicio

DEL CUADERNO AL SERVICIO

- El modelo se serializa con joblib en un único paquete: modelo, escalador y lista ordenada de variables. Así no se puede desalinear el orden de las columnas.
- El esquema de entrada es un modelo de pydantic: FastAPI valida tipos y campos obligatorios y devuelve un error 422 si falta una variable, sin código de validación propio.
- Documentación interactiva autogenerada en `/docs`; se prueba con el cliente de test de FastAPI y se lanza con uvicorn.
- Demo: la misma función expuesta como servidor **MCP** (transporte stdio), invocable desde un cliente como Claude Desktop sin escribir código de cliente.

API_SOFT_SENSOR.PY · EXTRACTO

```
import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel
class HoraProceso(BaseModel):
    iron_feed: float
    silica_feed: float
    almidon_medio: float
    amina_media: float
    caudal_pulpa: float
    ph_medio: float
    densidad: float
    aire_medio: float
    nivel_medio: float
bundle = joblib.load('soft_sensor_modelo.joblib')
modelo, scaler, features = (bundle['modelo'],
    bundle['scaler'], bundle['features'])
app = FastAPI(title="Soft sensor de silice - Semana 6")
@app.post("/predecir")
def predecir(hora: HoraProceso):
    ...
# uvicorn api_soft_sensor:app --reload
```

SEMANA 6 · DETALLE TÉCNICO (2/2)

Vigilar y explicar: MSPC como monitor de drift y SHAP

DRIFT

- El MSPC de la S4 se reajusta **solo con el tramo de train** y se aplica hora a hora sobre test: la tasa de alarma en producción frente a la de train mide si el proceso sigue pareciéndose a aquello con lo que se entrenó.
- Se separa lo que depende del umbral (recall, falsas alarmas) de lo que no (AUC).

SHAP PARA UN MODELO LINEAL

$$\varphi_j = \beta_j \, (x_j - E[x_j])$$

$$f(x) = E[f(X)] + \Sigma_j \, \varphi_j$$

Explicación local de una hora y ranking global por la media de $|\phi_j|$, que mezcla el coeficiente con cuánto varía la variable.

SEMANA6_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
masker = shap.maskers.Independent(X_tr,
                                  max_samples=X_tr.shape[0])
explainer = shap.LinearExplainer(modelo, masker)
sv = explainer(X_te[:1])        # una hora concreta de test
print(sv.base_values[0])                       # E[f(X)]
print(sv.base_values[0] + sv.values[0].sum())  # = predicción
for feat, v in sorted(zip(FEATURES, sv.values[0]),
                      key=lambda t: -abs(t[1])):
    print(f"{feat:15s} {v:+.4f}")
# comprobación: SHAP lineal = coeficiente × desviación
# respecto a la media de train
```

SEMANA 7 · AGENTES INTELIGENTES

De las reglas a un supervisor con LLM

QUÉ SE TRABAJA

- Satélite A: **agente de reglas** si-entonces sobre percentiles del proceso (el sistema experto clásico), evaluado con la precisión y el recall de la S3.
- Satélite B: **bucle de *tool-use* programado a mano** (patrón ReAct), primero con un ejemplo trivial y después con el *soft sensor* real; planificación con varias herramientas.
- LLM: SDK oficial `anthropic` contra un modelo de pesos abiertos servido localmente tras un proxy compatible (LiteLLM + Ollama). Las claves solo se leen del entorno.

PROYECTO ANCLA: AGENTE SUPERVISOR

- **F1**: tres herramientas, que son la predicción del *soft sensor*, `check_drift` (PCA + T^2/Q) y la explicación SHAP. Son los modelos de S2, S4 y S6, sin reentrenar.
- **F2**: *system prompt* con orden de razonamiento (deriva → explicación → recomendación), siempre condicionado a un operador humano.
- **F3**: verificación en tres horas reales: tranquila, problema de calidad sin deriva y deriva detectada.
- **F4**: AI Act del sistema como agente. *Ceiling*: dos agentes especializados y puente a la S8.

El agente recomienda; nunca actúa sobre la planta.

SEMANA 7 · DETALLE TÉCNICO (1/2)

El bucle de *tool-use*, programado a mano

PATRÓN REACT, PASO A PASO

1. Se envían los mensajes y la **descripción** de las herramientas (nombre, propósito, esquema JSON de entrada).
2. Si el modelo responde con `stop_reason == "tool_use"`, pide ejecutar una herramienta con unos argumentos.
3. La función la ejecuta **nuestro código**, nunca el modelo; el resultado vuelve como `tool_result` ligado a su `tool_use_id`.
4. Se repite hasta que el modelo termina (`end_turn`) con una respuesta en texto.

Primero con una herramienta trivial (sumar) para ver los bloques del protocolo; después con el *soft sensor*, y con varias herramientas que el modelo debe encadenar.

ESQUEMA DEL BUCLE, A PARTIR DEL SATÉLITE B

```
client = Anthropic(base_url=proxy_url, api_key=proxy_key)
mensajes = [{'role': 'user', 'content': pregunta}]
while True:
    resp = client.messages.create(model=modelo,
               max_tokens=1024, system=system_prompt,
               tools=herramientas, messages=mensajes)
    if resp.stop_reason != 'tool_use':
        break                              # respuesta final
    mensajes.append({'role': 'assistant',
                     'content': resp.content})
    resultados = []
    for b in resp.content:
        if b.type == 'tool_use':
            salida = funciones[b.name](**b.input)
            resultados.append({'type': 'tool_result',
                               'tool_use_id': b.id,
                               'content': str(salida)})
    mensajes.append({'role': 'user', 'content': resultados})
```

SEMANA 7 · DETALLE TÉCNICO (2/2)

El agente supervisor: herramientas, instrucciones y límites

DISEÑO

- Tres herramientas sobre artefactos ya existentes: el *soft sensor* de la S2 (joblib de la S6), el MSPC de la S4 y el explicador SHAP de la S6. No se entrena nada.
- La **descripción** de cada herramienta le dice al modelo cuándo usarla: `check_drift` "SIEMPRE antes de confiar en una predicción".
- *System prompt*: comprobar deriva primero, explicar después y recomendar solo al final, siempre condicionado a la decisión de un operador.
- Contraste con el satélite A: umbrales univariantes por percentiles frente a la detección multivariante de PCA + T^2/Q.

SEMANA7_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
_esquema_lecturas = {
    'type': 'object',
    'properties': {f: {'type': 'number'} for f in FEATURES},
    'required': list(FEATURES),
}
herramientas = [
  {'name': 'predict_silice',
   'description': 'Predice el porcentaje de silice ...',
   'input_schema': _esquema_lecturas},
  {'name': 'check_drift',
   'description': 'Comprueba si las lecturas ... se alejan '
                  'del comportamiento normal (MSPC T2 / Q). '
                  'Usar SIEMPRE antes de confiar en una '
                  'prediccion o recomendacion ...',
   'input_schema': _esquema_lecturas},
  {'name': 'explain_prediction',
   'description': 'Reparte la prediccion de silice entre '
                  'las 9 variables de proceso (SHAP) ...',
   'input_schema': _esquema_lecturas},
]
```

SEMANA 8 · OPTIMIZACIÓN METAHEURÍSTICA

De un agente que recomienda a un optimizador que calcula

QUÉ SE TRABAJA

- Búsqueda frente a optimización; métodos deterministas y metaheurísticos; computación evolutiva y recocido simulado (deck de teoría de 63 diapositivas).
- Satélite continuo: **Rosenbrock y Schaffer F6** con Nelder-Mead, GA propio y `differential_evolution` de scipy.
- Satélite combinatorio: array binario y **viajante** con codificación por permutación.
- Librería de algoritmos genéticos de la asignatura (`MAIIND_GA`), sin dependencias externas de optimización.

PROYECTO ANCLA, EN DOS PARTES

- **Parte A (continua)**: 3 consignas (almidón, amina, aire) con el resto fijo en una hora real fuera de spec; cotas en los percentiles 1–99 históricos.
- Descenso de gradiente proyectado (el bucle de la S2 recortado a las cotas) frente al GA. El óptimo se comprueba con el MSPC.
- **Parte B (combinatoria)**: qué subconjunto de las 9 variables usa el *soft sensor* (511 posibles), con el split temporal de la S2. Fuerza bruta, selección *forward* y *backward*, y GA.
- *Ceiling*: optimizar penalizando la distancia a los límites T^2/Q; puente al proyecto final.

Última pieza técnica del proyecto ancla. A partir de aquí empieza el mes de trabajo autónomo.

SEMANA 8 · DETALLE TÉCNICO (1/2)

El algoritmo genético de la asignatura

ESTRUCTURA DE `MAIIND_GA`

- Minimiza una función de aptitud cualquiera; la **codificación** la fijan la función que crea la población y la de cruce.
- En cada generación: selección por **torneo**; una fracción p_{cruce} se renueva por cruce, otra p_{mut} por mutación y el resto por reproducción directa; **elitismo** opcional.
- Cruce real: punto medio de los padres. Binario: monopunto. Permutación (viajante): recombinación por orden (OX), que siempre produce una ruta válida.
- Mutación: mezcla al 50 % con un individuo aleatorio, un salto grande sin búsqueda local fina.
- En el ancla: 200 individuos, 500 generaciones, $p_{cruce} = 0{,}75$, $p_{mut} = 0{,}2$, torneo de 3, con elitismo.

MAIIND_GA.PY Y SEMANA8_ANCLA_SOLUCION.IPYNB

```
def cruza_individuos_real_0_1(I1, I2):
    """Hijo = punto medio exacto entre los padres."""
    I1, I2 = np.squeeze(I1), np.squeeze(I2)
    return I1 + 0.5 * (I2 - I1)
def cruza_monopunto(I1, I2):
    punto_corte = np.random.randint(0, len(I1) - 1)
    I1, I2 = np.squeeze(I1), np.squeeze(I2)
    return np.concatenate((I1[:punto_corte],
                           I2[punto_corte:]))
# firma
# maiind_ga(eval_function, crea_indiv_function,
#           cruza_indiv_function, tam_indiv, tam_pop,
#           n_generaciones, p_cruce, p_mut,
#           tam_torneo, elitismo)
Mejores, FitMejor, FitPeor, FitMedio = maiind_ga(
    objetivo_01, crea_individuos_real_0_1,
    cruza_individuos_real_0_1,
    len(CONSIGNAS), 200, 500, 0.75, 0.2, 3, True)
```

SEMANA 8 · DETALLE TÉCNICO (2/2)

Ancla: gradiente proyectado y selección combinatoria

PARTE A · CONTINUA

$$z_{k+1} = \Pi_{[l,\,u]}(\, z_k - \alpha\, \nabla f\,)$$

El *soft sensor* es afín en las 3 consignas, así que en espacio estandarizado ∇f es el propio vector de coeficientes: constante. Sin cotas no hay mínimo; con la caja P1–P99, el óptimo está en una esquina. Para el GA, cada consigna se codifica en [0, 1] y se lleva a sus cotas. El punto final se pasa por el MSPC.

PARTE B · COMBINATORIA

Cromosoma binario de 9 bits (qué variables entran), $2^9 - 1 = 511$ subconjuntos y aptitud = R^2 en el split temporal de la S2. Se compara fuerza bruta (`itertools`), selección *forward* y *backward* y GA con cruce monopunto, contando evaluaciones.

SEMANA8_ANCLA_SOLUCION.IPYNB · EXTRACTO

```
coef_c = modelo.coef_[idx_consignas]
gradiente = coef_c      # espacio estandarizado: constante
lo_z = (lo - mean_c) / scale_c       # cotas P1 en z
hi_z = (hi - mean_c) / scale_c       # cotas P99 en z
def descenso_gradiente_proyectado(z0, grad, lo_z, hi_z,
                                  alpha, n_iter):
    z = z0.copy()
    historia = []
    for _ in range(n_iter):
        z = z - alpha * grad
        z = np.clip(z, lo_z, hi_z)   # proyección a la caja
        historia.append(z.copy())
    return z, historia
# GA: cada consigna se codifica en [0, 1]
def de_01_a_consignas(v01):
    v01 = np.clip(v01, 0, 1)
    return lo + v01 * (hi - lo)
```